Programming Assignment Report

Design and Analysis of Algorithms

Name: Shreyas S
Registration Number: 2411021061180

Problem Name: Real Estate Broker

Problem Statement

In the ancient city of Knossos, a real estate broker has a number of unsold houses and several interested clients. Each house has an **area** and a **minimum selling price**. Each client has a **minimum area requirement** and a **maximum price they are willing to pay**.

A client can buy a house only if:

- The **area of the house is greater than the client's required area**, and
- The **price of the house is less than or equal to the client's maximum budget**.

Constraints:

- Each **client can purchase only one house**.
- Each **house can be sold to only one client**.

The goal is to determine the **maximum number of houses that can be sold**.

HackerRank Problem Link:

https://www.hackerrank.com/challenges/real-estate-broker/problem

GitHub Submission Link:

https://github.com/shhoghg/DAA-LAB/tree/main/assignment%20DAA%201

Solution Approach

The problem can be represented as a **Bipartite Graph Matching problem**.

Two sets of nodes exist:

- **Clients**
- **Houses**

An edge exists between a client and a house if:

- The house area is **greater than the client's required area**, and
- The house price is **within the client's budget**.

The objective is to find the **maximum matching** so that the maximum number of houses can be sold.

To achieve this, we use the **Depth First Search (DFS) based Bipartite Matching algorithm**.

Algorithm Steps

1. Start the program.
2. Read the integers **n** (number of clients) and **m** (number of houses).
3. Store the area requirement and maximum budget for each client.
4. Store the area and minimum price for each house.
5. Create a graph where each client connects to houses that satisfy their conditions.
6. Initialize an array match[] to store the assigned client for each house.
7. Set all houses initially as **unassigned (-1)**.
8. For each client:
    - Reset the visited array.
    - Run **DFS** to find an available house.
9. If a house is free, assign it to the client.
10. If the house is already assigned, attempt to reassign its current owner to another house using DFS.
11. Count each successful assignment.
12. Print the maximum number of houses sold.
13. End the program.

Complete Java Code

```
import java.util.*;

public class Main {

    static List<Integer>[] graph;
    static int[] match;
    static boolean[] visited;

    static boolean dfs(int u) {
        for (int v : graph[u]) {
            if (!visited[v]) {
                visited[v] = true;
                if (match[v] == -1 || dfs(match[v])) {
                    match[v] = u;
                    return true;
                }
            }
```

```
    }
    return false;
}

public static void main(String[] args) {
    Scanner sc = new Scanner(System.in);

    int n = sc.nextInt();
    int m = sc.nextInt();

    int[] x = new int[n];
    int[] q = new int[n];

    for (int i = 0; i < n; i++) {
        x[i] = sc.nextInt();
        q[i] = sc.nextInt();
    }

    int[] y = new int[m];
    int[] p = new int[m];

    for (int i = 0; i < m; i++) {
        y[i] = sc.nextInt();
        p[i] = sc.nextInt();
    }

    graph = new ArrayList[n];
    for (int i = 0; i < n; i++)
        graph[i] = new ArrayList<Integer>();

    for (int i = 0; i < n; i++) {
        for (int j = 0; j < m; j++) {
            if (y[j] > x[i] && p[j] <= q[i]) {
                graph[i].add(j);
            }
        }
    }

    match = new int[m];
    Arrays.fill(match, -1);

    int result = 0;

    for (int i = 0; i < n; i++) {
        visited = new boolean[m];
        if (dfs(i))
            result++;
    }

    System.out.println(result);
```

```
    }
}
```

Time Complexity

Graph Construction:

O(n × m)

Matching using DFS:

O(n × m)

Overall Time Complexity:

O(n × m)

Where:

- **n = number of clients**
- **m = number of houses**

Observation / Result

The program successfully determines the **maximum number of houses that can be sold** by matching clients with houses that meet their requirements.

Key observations:

- The problem is solved using **Bipartite Graph Matching**.
- DFS is used to find **augmenting paths** for reassigning houses if needed.
- Each client and house is matched **only once**, satisfying all constraints.
- The algorithm ensures the **maximum possible transactions between clients and houses**.

Thus, the final output represents the **maximum number of houses that can be sold**.